

LAPORAN PRAKTIKUM
PEMROGRAMAN WEB – GIK1JAB3
Modul 6: Pemrograman Berorientasi Objek (OOP) pada PHP

CHALISA GAVRILLA AZHARI

707012500127

D4 SIKC 49-04

PROGRAM STUDI D4 SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

A. Tujuan Praktikum

1. Memahami konsep dasar Pemrograman Berorientasi Objek (OOP) dalam PHP.
2. Menggunakan tipe data object dalam PHP.
3. Membuat dan menggunakan class serta object.
4. Memahami konsep constructor dan destructor.
5. Menerapkan konsep inheritance dalam OOP PHP.

B. Alat dan Bahan

1. Komputer dengan sistem operasi Windows, macOS, atau Linux.
2. Web server seperti XAMPP.
3. Editor teks atau IDE seperti VS Code, Sublime Text, atau PHPStorm.
4. Browser web (Google Chrome, Mozilla Firefox, atau lainnya).

C. Langkah-langkah praktikum

1. Gunakan file praktikum sebelumnya
2. Buat folder Pekan-06 dalam folder WebPro4904 untuk memudahkan pengumpulan pada github.
3. Lanjutkan pengerjaan sebelumnya dengan menambahkan OOP pada Java
4. Setelah berhasil, lalu kumpulkan.

D. Hasil dan Dokumentasi Program

1. Koneksi

```

1 <?php
2 class Database { // Kelas untuk mengelola koneksi database
3     private $host = "localhost"; // Sesuaikan dengan host database
4     private $username = "root"; // Sesuaikan dengan username database
5     private $password = ""; // Sesuaikan dengan password database
6     private $database = "lifetrack"; // Sesuaikan dengan nama database
7     public $conn; // Variabel untuk menyimpan koneksi yang akan digunakan oleh file lain
8
9     public function __construct() { // Konstruktor untuk membuat koneksi saat objek dibuat
10        // Menggunakan mysqli secara OOP
11        $this->conn = new mysqli($this->host, $this->username, $this->password, $this->database); // Membuat koneksi ke database
12
13        if ($this->conn->connect_error) { // Cek koneksi
14            die("Koneksi gagal: " . $this->conn->connect_error); // Jika koneksi gagal, tampilkan pesan error
15        }
16    }
17 }
18
19 $db = new Database(); // Membuat objek database yang akan mengelola koneksi
20 $conn = $db->conn; // Variabel ini digunakan oleh file lain
21 ?>

```

2. Registrasi

```

1 <?php
2 include "koneksi.php"; // Menghubungkan ke file koneksi untuk akses database
3
4 class UserAuth { // Kelas untuk mengelola pendaftaran pengguna
5     private $db; // Variabel untuk menyimpan koneksi database
6     public function __construct($connection) { // Konstruktor untuk menerima koneksi database saat objek dibuat
7         $this->db = $connection; // Menyimpan koneksi database ke dalam variabel kelas
8     }
9
10    public function daftar($data) { // Fungsi untuk mendaftarkan pengguna baru, menerima data dari form
11        $role = $data['role'] ?? ""; // Mengambil peran dari data, jika tidak ada maka kosong
12        $name = trim($data['name'] ?? ""); // Mengambil nama dari data, jika tidak ada maka kosong, dan menghapus spasi di awal/akhir
13        $email = trim($data['email'] ?? ""); // Mengambil email dari data, jika tidak ada maka kosong, dan menghapus spasi di awal/akhir
14        $password = trim($data['password'] ?? ""); // Mengambil password dari data, jika tidak ada maka kosong, dan menghapus spasi di awal/akhir
15
16        if ($role == "" || $name == "" || $email == "" || $password == "") { // Cek jika ada field yang kosong
17            return ["status" => "danger", "msg" => "Semua field wajib diisi!"]; // Jika ada yang kosong, kembalikan status error dan pesan
18        }
19
20        $hashed_password = password_hash($password, PASSWORD_DEFAULT); // Hash password menggunakan algoritma default (bcrypt) untuk keamanan
21        $query = "INSERT INTO users (role, name, email, password) VALUES ('$role', '$name', '$email', '$hashed_password')"; // Query SQL untuk memasukkan data pengguna baru ke dalam tabel users
22
23        // Eksekusi query dan cek hasilnya
24        if (mysqli_query($this->db, $query)) { // Jika query berhasil, kembalikan status sukses dan pesan
25            return ["status" => "success", "msg" => "Akun berhasil dibuat! Silakan login."];
26        } else { // Jika query gagal, kembalikan status error dan pesan dengan detail error dari database
27            return ["status" => "danger", "msg" => "Gagal: " . mysqli_error($this->db)];
28        }
29    }
30 }
31
32 $auth = new UserAuth($conn); // Membuat objek UserAuth dengan koneksi database yang sudah dibuat di koneksi.php
33 $message = ""; $type = ""; $role = ""; $name = ""; $email = ""; // Inisialisasi variabel untuk menyimpan pesan, tipe pesan, dan nilai form agar tidak error saat form pertama dimuat
34
35 if ($_SERVER['REQUEST_METHOD'] == "POST") { // Cek jika form disubmit dengan metode POST
36     $role = $_POST['role'] ?? ""; // Mengambil nilai role dari form, jika tidak ada maka kosong
37     $name = $_POST['name'] ?? ""; // Mengambil nilai name dari form, jika tidak ada maka kosong
38     $email = $_POST['email'] ?? ""; // Mengambil nilai email dari form, jika tidak ada maka kosong
39     $result = $auth->daftar($_POST); // Memanggil fungsi daftar dari objek UserAuth dengan data yang dikirimkan dari form, dan menyimpan hasilnya ke dalam variabel result
40     $message = $result['msg']; // Mengambil pesan dari hasil fungsi daftar dan menyimpan ke dalam variabel message untuk ditampilkan di form
41     $type = $result['status']; // Mengambil status dari hasil fungsi daftar dan menyimpan ke dalam variabel type untuk menentukan jenis alert yang akan ditampilkan di form
42
43     // Jika pendaftaran berhasil, redirect ke halaman login setelah 2 detik
44     if ($type == "success") { header("refresh:2;url=login.php"); }
45 }
46 ?>

```

3. Login

```

1 <?php
2 include "koneksi.php"; // Menghubungkan ke file koneksi untuk akses database
3 session_start(); // Memulai sesi untuk menyimpan informasi login
4
5 class LoginSystem { // kelas untuk mengelola proses login
6     private $db; // Variabel untuk menyimpan koneksi database
7     public function __construct($connection) { $this->$db = $connection; } //
8
9     public function login($role, $email, $password) { // Fungsi untuk melakukan login, menerima peran, email, dan password dari form
10         $email = mysqli_real_escape_string($this->$db, $email); // Melindungi dari SQL Injection dengan memescape input email
11         $query = mysqli_query($this->$db, "SELECT * FROM users WHERE email='$email' AND role='$role'"); // Query untuk mencari pengguna dengan email dan peran yang sesuai
12         $user = mysqli_fetch_assoc($query); // Pengambil data pengguna dari hasil query
13
14         if ($user && password_verify($password, $user['password'])) { // Cek jika pengguna ditemukan dan password cocok dengan hash di database
15             $SESSION['login'] = true; // Menandai bahwa pengguna sudah login
16             $SESSION['name'] = $user['name']; // Menyimpan nama pengguna di sesi untuk ditampilkan di dashboard
17             $SESSION['role'] = $user['role']; // Menyimpan peran pengguna di sesi untuk kontrol akses di halaman lain
18             return true; // Login berhasil
19         }
20         return false; // Login gagal jika pengguna tidak ditemukan atau password salah
21     }
22 }
23
24 $auth = new LoginSystem($conn); // Membuat objek LoginSystem dengan koneksi database yang sudah dibuat di koneksi.php
25 $message = ""; $type = ""; $email = $_COOKIE['email'] ?? ""; $role = ""; // Inisialisasi variabel untuk menyimpan pesan, tipe pesan, email yang diisi sebelumnya (dari cookie), dan peran yang dipilih sebelumnya agar tidak error saat form pertama dimuat
26
27 if ($_SERVER['REQUEST_METHOD'] == "POST") { // Cek jika form disubmit dengan metode POST
28     $role = $_POST['role']; // Menambil nilai role dari form
29     if ($auth->login($role, $_POST['email'], $_POST['password'])) { // Memanggil fungsi login dari objek LoginSystem dengan data yang dikirimkan dari form, dan cek jika login berhasil
30         $message = "Login berhasil! Mengalihkan..."; $type = "success"; // Jika login berhasil, simpan email di cookie selama 30 hari untuk mengisi otomatis saat login berikutnya
31         header("refresh:30,url=dashboard.php"); // Redirect ke dashboard setelah 3 detik
32     } else { // Jika login gagal, tampilkan pesan error
33         $message = "Email atau Password salah!"; $type = "danger";
34     }
35 }
36 }

```

4. Dashboard

```

1 <?php
2 include "koneksi.php"; // Menghubungkan ke file koneksi untuk akses database
3
4 session_start(); // Memulai sesi untuk menyimpan informasi login
5 if (!isset($SESSION['login'])) { // Cek jika pengguna belum login, jika belum maka redirect ke halaman login
6     header("Location: login.php"); // Redirect ke halaman login jika belum login
7     exit; // Hentikan eksekusi script setelah redirect
8 }
9
10 $nama_dokter = "Dr. Andi"; // Variabel untuk menyimpan nama dokter yang akan ditampilkan di sidebar
11 $profile_pic = "calm pfp.jpg"; // Variabel untuk menyimpan path foto profil dokter
12 $current_page = basename($_SERVER['PHP_SELF']); // Variabel untuk menyimpan nama file halaman saat ini, digunakan untuk menandai menu aktif
13 ?>

```

5. Layanan Tenaga Medis

```

1 <?php
2 session_start(); // Memulai sesi untuk menyimpan informasi login
3 include 'koneksi.php'; // Menghubungkan ke file koneksi untuk akses database
4
5 // Cek login
6 if (!isset($_SESSION['login'])) { // Cek jika pengguna belum login, jika belum maka redirect ke halaman login
7     header("Location: login.php"); // Redirect ke halaman login jika belum login
8     exit; // Hentikan eksekusi script setelah redirect
9 }
10
11 class Layanan { // Kelas untuk mengelola layanan informasi kesehatan
12     private $db; // Variabel untuk menyimpan koneksi database
13     public function __construct($connection) { // Konstruktor untuk menerima koneksi database saat objek dibuat
14         $this->db = $connection; // Menyimpan koneksi database ke dalam variabel kelas
15     }
16
17     public function tampilSemua() { // Fungsi untuk mengambil semua data informasi kesehatan dari database
18         return mysql_query($this->db, "SELECT * FROM informasi_medis"); // Mengembalikan hasil query untuk diolah di bagian tampilan
19     }
20
21     public function hapus($id) { // Fungsi untuk menghapus data informasi kesehatan berdasarkan ID
22         $id = mysql_real_escape_string($this->db, $id); // Melindungi dari SQL Injection dengan memberikan input ID
23         return mysql_query($this->db, "DELETE FROM informasi_medis WHERE id=$id"); // Eksekusi query untuk menghapus data berdasarkan ID
24     }
25
26     public function simpan($data, $file) { // Fungsi untuk menyimpan data informasi kesehatan, baik untuk menambah data baru maupun mengupdate data yang sudah ada, menerima data dari form dan file yang diupload
27         $judul = mysql_real_escape_string($this->db, $data['judul']); // Melindungi dari SQL Injection dengan memberikan input judul
28         $kategori = mysql_real_escape_string($this->db, $data['kategori']); // Melindungi dari SQL Injection dengan memberikan input kategori
29         $deskripsi = mysql_real_escape_string($this->db, $data['deskripsi']); // Melindungi dari SQL Injection dengan memberikan input deskripsi
30         $id = isset($data['editindex']) ? mysql_real_escape_string($this->db, $data['editindex']) : ""; // Melindungi dari SQL Injection dengan memberikan input ID untuk edit, jika ada
31
32         $namafile = $file['name']; // Nama file yang diupload
33         $tmpfile = $file['tmp_name']; // Lokasi sementara file yang diupload
34         $path = "upload/" . $namafile; // Path tujuan untuk menyimpan file yang diupload, pastikan folder 'upload' sudah ada dan memiliki izin yang benar
35
36         // Upload file jika ada
37         if (!empty($tmpfile)) { // Cek jika ada file yang diupload
38             move_uploaded_file($tmpfile, $path); // Pindahkan file dari lokasi sementara ke folder tujuan, pastikan folder 'upload' sudah ada dan memiliki izin yang benar
39         }
40
41         if (!empty($id)) { // Cek jika ID ada, berarti ini adalah update data, jika tidak maka ini adalah tambah data baru
42             // Update data
43             if (!empty($namafile)) { // Jika ada file baru yang diupload saat edit, update juga field file di database
44                 return mysql_query($this->db, "UPDATE informasi_medis SET judul='$judul', kategori='$kategori', deskripsi='$deskripsi', file='$namafile' WHERE id=$id"); // Eksekusi query untuk mengupdate data berdasarkan ID, termasuk update file jika ada
45             } else { // Jika tidak ada file baru yang diupload saat edit, jangan update field file di database
46                 return mysql_query($this->db, "UPDATE informasi_medis SET judul='$judul', kategori='$kategori', deskripsi='$deskripsi' WHERE id=$id");
47             }
48         } else {
49             // Tambah data baru
50             return mysql_query($this->db, "INSERT INTO informasi_medis (judul, kategori, deskripsi, file) VALUES ('$judul', '$kategori', '$deskripsi', '$namafile')"); // Eksekusi query untuk menambah data baru ke database, termasuk nama file yang diupload
51         }
52     }
53
54     public function ambilSatu($id) { // Fungsi untuk mengambil satu data informasi kesehatan berdasarkan ID, digunakan untuk menampilkan data yang akan diedit di form
55         $id = mysql_real_escape_string($this->db, $id); // Melindungi dari SQL Injection dengan memberikan input ID
56         $res = mysql_query($this->db, "SELECT * FROM informasi_medis WHERE id=$id"); // Eksekusi query untuk mengambil data berdasarkan ID
57         return mysql_fetch_assoc($res); // Mengembalikan data sebagai array asosiatif untuk ditampilkan di form edit, jika data ditemukan maka akan berisi data, jika tidak ditemukan maka akan bernilai null
58     }
59 }
60
61 $layananObj = new Layanan($conn); // Membuat objek layanan dengan koneksi database yang sudah dibuat di koneksi.php
62
63 // Inisialisasi variabel agar tidak error saat form pertama dibuat
64 $editindex = null;
65 $judul = ""; // Variabel untuk menyimpan data yang akan diedit, jika ada
66 $kategori = "Perawatan & Medis"; // Variabel untuk menyimpan data yang akan diedit, jika ada, dengan default kategori pertama
67 $deskripsi = ""; // Variabel untuk menyimpan data yang akan diedit, jika ada
68
69 // Cek aksi form
70 if (isset($_GET['hapus'])) { // Panggil fungsi hapus dengan ID yang dikirimkan melalui query string
71     $layananObj->hapus($_GET['hapus']); // Panggil fungsi hapus dengan ID yang dikirimkan melalui query string
72     header("Location: layanan_tenagamedis.php"); // Redirect ke halaman yang sama untuk melihat perubahan
73     exit; // Hentikan eksekusi script setelah redirect
74 }
75
76 // Cek aksi aksi data (ambil data)
77 if (isset($_GET['edit'])) { // Ambil data yang akan diedit berdasarkan ID yang dikirimkan melalui query string
78     $dataedit = $layananObj->ambilSatu($_GET['edit']); // Ambil data yang akan diedit berdasarkan ID yang dikirimkan melalui query string
79     if ($dataedit) { // Jika data ditemukan, isi variabel untuk menampilkan di form edit
80         $editindex = $dataedit['id']; // Simpan ID yang sedang diedit untuk digunakan di form
81         $judul = $dataedit['judul']; // Ambil data judul untuk ditampilkan di input saat edit
82         $kategori = $dataedit['kategori']; // Ambil data kategori untuk menampilkan pilihan yang sesuai di dropdown saat edit
83         $deskripsi = $dataedit['deskripsi']; // Ambil data deskripsi untuk ditampilkan di textarea saat edit
84     }
85 }
86
87 // Cek aksi aksi simpan/update
88 if ($_SERVER['REQUEST_METHOD'] === "POST") { // Panggil fungsi simpan dengan data dari form dan file yang diupload
89     $layananObj->simpan($_POST, $_FILES['file']); // Panggil fungsi simpan dengan data dari form dan file yang diupload
90     header("Location: layanan_tenagamedis.php"); // Redirect ke halaman yang sama untuk melihat perubahan
91     exit; // Hentikan eksekusi script setelah redirect
92 }
93
94 $dataInformasi = $layananObj->tampilSemua(); // Ambil semua data informasi untuk ditampilkan di tabel
95 }

```

E. Analisis dan Pembahasan

Penerapan konsep Object-Oriented Programming (OOP) pada proyek LifeTrack bertujuan untuk merapikan struktur kode dengan mengelompokkan fungsi-fungsi database dan logika medis ke dalam wadah khusus bernama *Class*. Perubahan ini berhasil mengatasi masalah error "variabel tidak dikenal" melalui sistem inisialisasi variabel di awal file, sehingga tampilan form menjadi bersih dan profesional. Selain itu, penggunaan metode OOP meningkatkan keamanan data karena input dari pengguna disaring terlebih dahulu sebelum diproses ke database, yang menjamin fitur tambah, ubah, dan hapus data bagi tenaga medis berjalan lebih stabil dan terorganisir.

F. Kesimpulan

Dapat disimpulkan bahwa transisi dari kode prosedural ke OOP memberikan pondasi yang lebih kuat dan sistematis bagi aplikasi LifeTrack dalam mengelola informasi kesehatan pasien secara dinamis. Struktur database yang ada tetap dapat digunakan sepenuhnya tanpa perubahan, sementara kode program menjadi lebih mudah dikelola untuk pengembangan fitur di masa depan. Dengan sistem yang lebih rapi dan bebas dari gangguan error teknis pada antarmuka, aplikasi ini kini lebih siap digunakan oleh tenaga medis untuk menunjang kualitas hidup pasien pasca-perawatan secara real-time.

G. Lampiran berisi Link repositori GitHub

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/ChalisaGavrillaAzhari-707012500127>